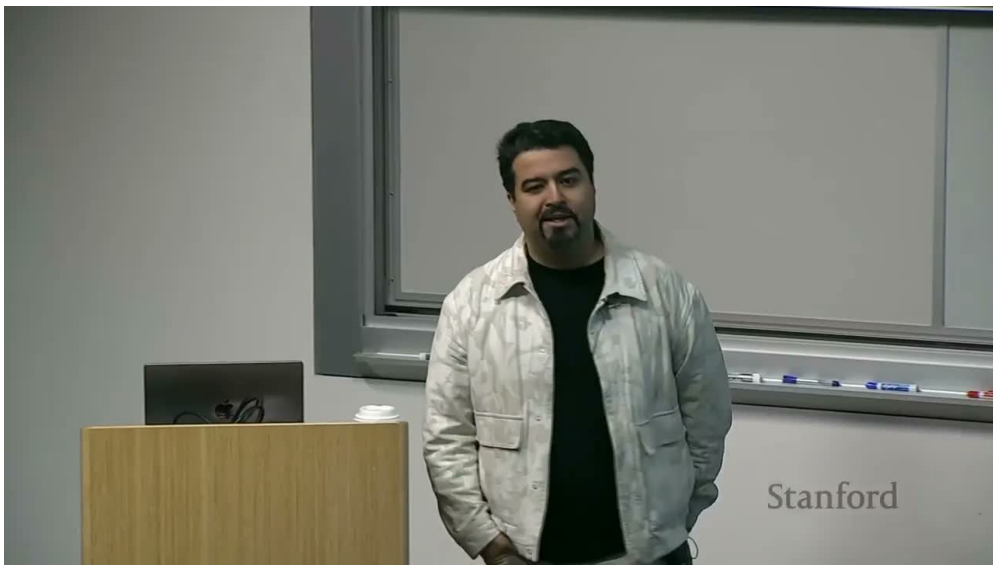


课程笔记

CS231N 2025 第 4 讲：神经网络与反向传播

Codex Harness Worker 1

2026-04-15



视频作者/频道：Stanford Online

发布日期：2025-09-02

视频时长：1:16:46

视频链接：<https://www.youtube.com/watch?v=25zD5qJHYsk>

目录

1	从线性模型到多层感知机	2
1.1	本章小结	2
2	深度、宽度与表示能力	2
2.1	本章小结	3
3	为什么必须高效求梯度	3
3.1	本章小结	3
4	反向传播的核心：链式法则在计算图上的执行	3
4.1	本章小结	5
5	局部梯度模式与工程实现	5
5.1	本章小结	6
6	总结与延伸	6
6.1	拓展阅读	6

1 从线性模型到多层感知机

Lecture 4 的第一步, 是解释为什么线性分类器不够。即使上一讲已经有了损失函数与优化器, 只要模型本身仍是线性的, 它就无法表达复杂的类别边界。多层感知机 (multi-layer perceptron, MLP) 通过在仿射变换之间插入非线性激活函数, 把简单线性映射提升为高度灵活的分段线性函数族。

一个典型的两层网络可以写成

$$h = \sigma(W_1x + b_1), \quad s = W_2h + b_2.$$

其中, x 是输入, W_1, b_1 是第一层参数, h 是隐藏表示, $\sigma(\cdot)$ 是非线性激活, s 是输出分数。若选择 ReLU, 便有 $\sigma(z) = \max(0, z)$ 。这里的符号解释非常重要: 它说明网络先构造中间表示, 再用输出层完成决策。

Neural networks: why is max operator important?

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1x)$

The function $\max(0, z)$ is called the activation function.

Q: What if we try to build a neural network without one?

$$f = W_2W_1x$$

图 1: 两层感知机通过非线性激活把线性分类器升级为分段线性函数族; “max” 或 ReLU 是表达能力跃迁的关键。

为什么非线性不可省略

如果每一层都只是线性变换, 那么多层连乘仍然等价于一层线性变换。只有引入 ReLU、sigmoid、tanh 等非线性, 深度才真正增加函数表达能力。

神经网络相较于线性分类器的本质升级, 在于引入了可组合的非线性。两层网络已经能表达远比线性模型更复杂的函数族。

1.1 本章小结

2 深度、宽度与表示能力

网络更深、更宽, 通常意味着更大的表示容量 (capacity)。但课程并没有把这个结论说成 “越大越好”, 而是提醒读者: 更大容量同时意味着更高的过拟合风险、更难的优化问题以及更强的正则

化需求。也就是说，神经网络之所以强，不是因为参数多本身，而是因为这些参数在合适的的数据、目标函数和训练机制下，能够形成有用表示。

教材化理解时，可以把隐藏层看成逐步构造中间特征的过程：第一层可能对局部模式敏感，第二层组合成更抽象的部件，最后一层对类别决策负责。虽然这种解释在不同模型中并不总是字面成立，但它提供了理解深网络的第一性直觉：深度允许逐层重写表示。

深度与宽度提高了模型容量，但也带来了优化与泛化的新困难。网络真正学到的是中间表示，而不只是最后一层分类器。

2.1 本章小结

3 为什么必须高效求梯度

一旦模型变成多层非线性结构，参数数量就迅速膨胀。若仍像上一讲那样用有限差分逐个参数近似梯度，计算代价会灾难性增长。于是课程自然转入一个核心问题：给定一个由许多简单运算组成的复杂函数，如何系统而高效地求出每个参数的梯度？

答案就是把网络看成计算图（computational graph）。前向传播负责计算每个节点的数值，反向传播则沿图结构把损失对输出的梯度逐步传回输入与参数。这个思想极其强大，因为它把求导从“手写一个巨大公式”变成了“为每个基本算子定义局部梯度规则，然后通过链式法则自动组合”。

深模型训练的关键瓶颈不是“有没有目标函数”，而是“能否高效得到所有参数的梯度”。计算图把复杂函数拆成局部算子，为反向传播提供了统一执行框架。

3.1 本章小结

4 反向传播的核心：链式法则在计算图上的执行

课程先从标量例子入手，因为这能把链式法则讲到最透明。设某个中间变量 $q = xy$ ，输出为 $f = q + z$ ，那么损失对输入的梯度来自局部导数与上游梯度的连乘。如果已知 $\frac{\partial L}{\partial f}$ ，则有

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x}, \quad \frac{\partial L}{\partial y} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y}.$$

其中， $\frac{\partial L}{\partial f}$ 称为上游梯度（upstream gradient）， $\frac{\partial f}{\partial q}$ 、 $\frac{\partial q}{\partial x}$ 等则是局部梯度（local gradient）。

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

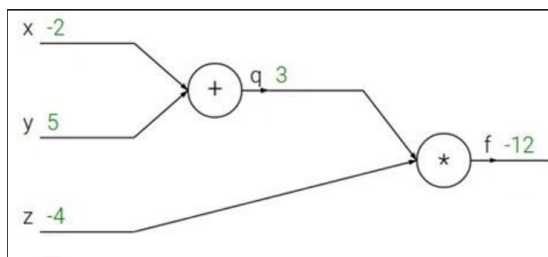


图 2: 标量计算图示例: 先做前向值传播, 再沿图结构逐边乘上局部梯度, 这就是反向传播的工作方式。

Backpropagation: a simple example

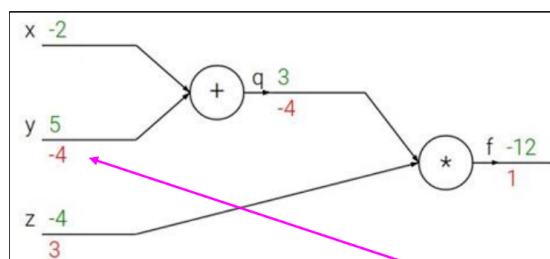
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient Local gradient

$$\frac{\partial f}{\partial y}$$

图 3: 上游梯度 (upstream gradient) 与局部梯度 (local gradient) 的乘积给出当前节点的梯度贡献。

这套机制最重要的地方在于可复用: 无论整个网络多复杂, 每个节点只关心两件事, 第一是前向值, 第二是局部梯度。前者在前向传播时缓存下来, 后者在反向传播时与上游梯度相乘。把这种局部规则串起来, 就得到了全局梯度。

反向传播不是神秘技巧, 而是链式法则在计算图上的系统执行。每个节点只需知道自己的前向值和局部梯度, 就能参与全局求导。

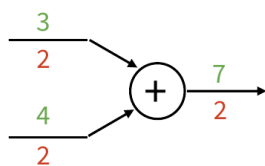
4.1 本章小结

5 局部梯度模式与工程实现

课程随后总结了几个最重要的局部梯度模板。对加法节点，梯度会被原样分发到每个输入；对乘法节点，每个输入会乘上“另一个输入”的数值；对 max/ReLU 节点，只有赢得最大值的分支会接收梯度。看似琐碎的这些规则，实际上组成了几乎所有神经网络层的底层求导积木。

Patterns in gradient flow

add gate: gradient distributor



mul gate: “swap multiplier”

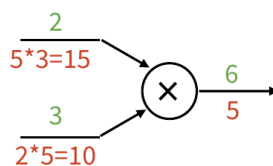
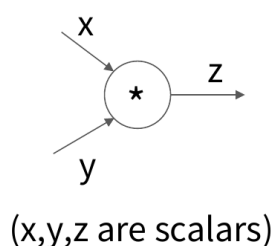


图 4: 不同算子对应不同梯度流模式：加法分发梯度，乘法交换乘子，max 只把梯度传给获胜分支。

工程实现上，最重要的设计是 forward/backward API：前向函数返回输出并缓存必要中间量，反向函数接收上游梯度并返回对输入与参数的梯度。把每个层封装成这种接口之后，神经网络训练就变成了“缓存前向中间量 + 统一反向接口”的工程问题。

Modularized implementation: forward / backward API

Gate / Node / Function object: Actual PyTorch code



```
class Multiply(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, y):
        ctx.save_for_backward(x, y)  # Need to cache some values for use in backward
        z = x * y
        return z
    @staticmethod
    def backward(ctx, grad_z):  # Upstream gradient
        x, y = ctx.saved_tensors
        grad_x = y * grad_z  # dz/dx * dL/dz
        grad_y = x * grad_z  # dz/dy * dL/dz
        return grad_x, grad_y  # Multiply upstream and local gradients
```

Stanford CS231n 10th Anniversary

Lecture 4 - 115

April 10, 2025

图 5: 把每个层封装成 forward/backward API 后, 神经网络训练就变成“缓存前向中间量 + 统一反向接口”的工程问题。

为了把“实现”说得更具体, 可以把一层的训练接口写成如下伪代码:

```
1 out, cache = layer.forward(x)
2 dx, dw, db = layer.backward(dout, cache)
3 params['W'] -= lr * dw
4 params['b'] -= lr * db
```

Listing 1: 单层 forward/backward 接口的伪代码

局部梯度规则与模块化实现共同构成了现代自动求导框架的基础。理解这一节之后, 再看 PyTorch 的 autograd, 就不会把它当成黑箱。

5.1 本章小结

6 总结与延伸

Lecture 4 完成了从“可优化的线性模型”到“可训练的深层非线性模型”的关键跨越。它先解释了神经网络为什么需要非线性, 再说明反向传播为什么能够让大规模参数学习真正可行。若没有这一讲, 后续卷积网络、自注意力、多模态模型的训练都会失去基础。

从教材的组织上看, 这一讲把深度学习最核心的训练机制讲清了: 前向传播负责计算值, 反向传播负责沿计算图传递梯度, 优化器负责用梯度更新参数。这三者共同构成现代深度模型训练的主轴。下一讲进入卷积网络时, 我们只是把“层的种类”换掉, 而不是把训练逻辑推倒重来。

6.1 拓展阅读

- Backprop
- Why Momentum Really Works

- Backpropagation Blog